

Group 05 - A.L.B.E.R.T.

Table of Contents

1. Group	2
2. Introduction.....	2
2.1. Idea.....	2
2.2. Goals	2
3. Technical implementation.....	3
3.1. Wind turbine	3
3.1.1. Description.....	3
3.1.2. Challenges.....	3
3.1.3. Components	4
3.1.4. 3D-printed parts	12
3.1.5. Wind turbine and Full Wave Rectifier circuit diagram	17
3.2. Wind direction sensor / Wind vane.....	17
3.2.1. Description.....	17
3.2.2. Challenges.....	17
3.2.3. Components	18
3.2.4. 3D-printed parts	20
3.3. Monitoring system	24
3.3.1. Description.....	24
3.3.2. Challenges.....	24
3.3.3. Components	24
3.3.4. 3D-printed parts	29
3.3.5. Circuit diagram	29
3.4. Wind turbine box	30
3.5. Development pictures	30
4. Result	31
4.1. Images	31
4.2. Overview video	31
5. Commented code	32
6. API links	39
7. Improvement ideas.....	39
8. Sources.....	39

1. Group

Members:

Name	Major
Jussi Nurmi	Electrical Energy Technology
Eino Hannunen	Electronics and Electrophysics
Tsvetomir Zhivkov	Computer Science
Alexey Volkov	Electrical Energy Technology

2. Introduction

2.1. Idea

"Aerodynamical Logical Brutal Electrical Rotating Turbine" (= A.L.B.E.R.T)

Our team consisted of students interested in energy technology. We chose to develop a small-scale wind turbine model to gain practical experience in a wide range of topics, including electronics, programming, power engineering, and power electronics.

The project was particularly appealing due to its modular structure. Components such as energy storage, a monitoring system, and grid integration could be added or omitted as the project progressed, allowing the scope to be adjusted according to the available time. This flexibility gave us confidence that we would be able to demonstrate a functional system by the end of the course.

2.2. Goals

- Build a small wind turbine prototype consisting of all key components (tower, generator and rotor).
- Enable the nacelle to rotate according to the wind direction (maximizing the conversion of wind energy into electricity).
- Design a monitoring system that monitors the state of the wind turbine.
- Rectify the generated voltage.
- Store the generated energy. (**not implemented**)

3. Technical implementation

3.1. Wind turbine

3.1.1. Description

The wind turbine consists of 3D-printed tower and nacelle. The nacelle includes a 3-phase generator, gearbox and a shaft that drives the generator due to the rotation of the rotor. The nacelle is rotated by a stepper motor connected to a power source consisting of six 1.2 V rechargeable batteries. The nacelle rotation range is approximately 240 degrees.

The core of the wind turbine control system is an ESP32 microcontroller, which is supplied with 3.3V operating voltage through a step-down converter. The microcontroller transmits data on the generated voltage and the status of the stepper motor's power supply to a dedicated server over the network. Both signals are measured using voltage sensors. The microcontroller interfaces with a wind direction sensor and a stepper motor driven by the TMC2209 driver. Stepper motor's operation is controlled using feedback from the wind vane's Hall-sensor.

3.1.2. Challenges

Generator cables:

To enable nacelle rotation, we had to consider the possibility of generator's cables becoming twisted. As a solution, we limited the nacelle's rotation region using two microswitches. The microswitches also allowed the nacelle to perform a calibration procedure when the turbine is powered on.

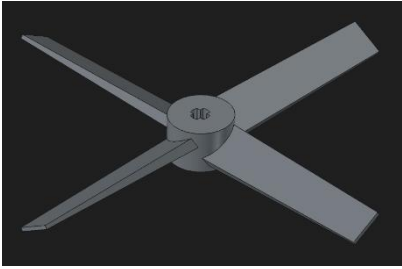
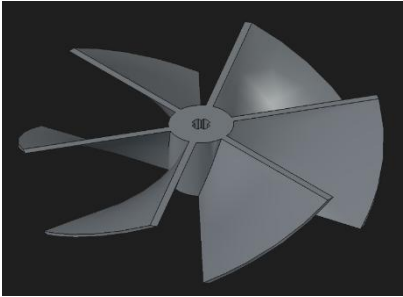
3-phase generator rectifier and voltage sensors

According to the project objectives, the electrical power generated by the turbine had to be utilized, which required a three-phase full wave rectifier. The basic rectifier design was readily available from online sources. The implemented circuit used six diodes and one capacitor to rectify the generator output and smooth the resulting DC voltage.

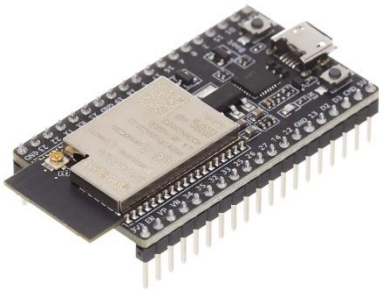
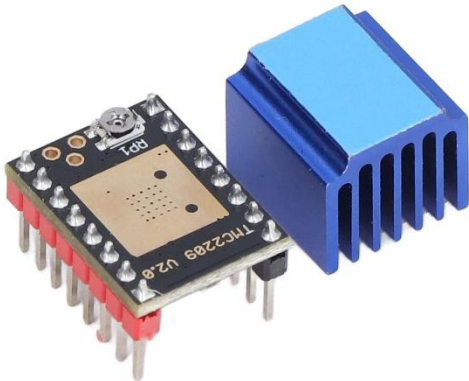
The generated voltage and the battery capacity had to be monitored, which required voltage sensors. We designed a simple voltage sensor based on a voltage divider consisting of two resistors and a small capacitor. We took into account the microcontroller input voltage limits, power consumption and source output impedance.

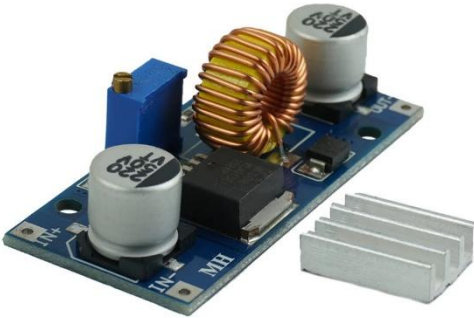
Rotor

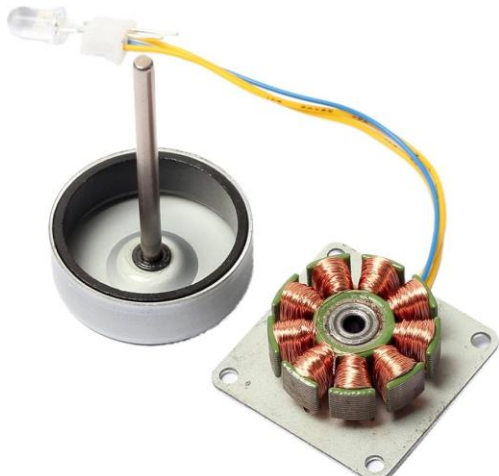
Our initial developed version of the rotor did not generate enough torque to drive the generator through the gearbox. To address this issue, we decided to use a rotor design based on a PC fan, which performed well during testing. The new rotor increased the turbine's generated power output by approximately a factor of ten.

Old rotor	New rotor
	

3.1.3. Components

Component	Amount	Image
ESP32 WROVER-IE	1	
TMC2209 Stepper motor driver	1	

Stepper motor	1	
Step Down Converter	1	
USB-connector (female)	1	

Battery case (6x AA)	1	
Rechargeable batteries	6	
3-phase generator	1	

Microswitch	2	
Switch	1	
100nF capacitor	2	

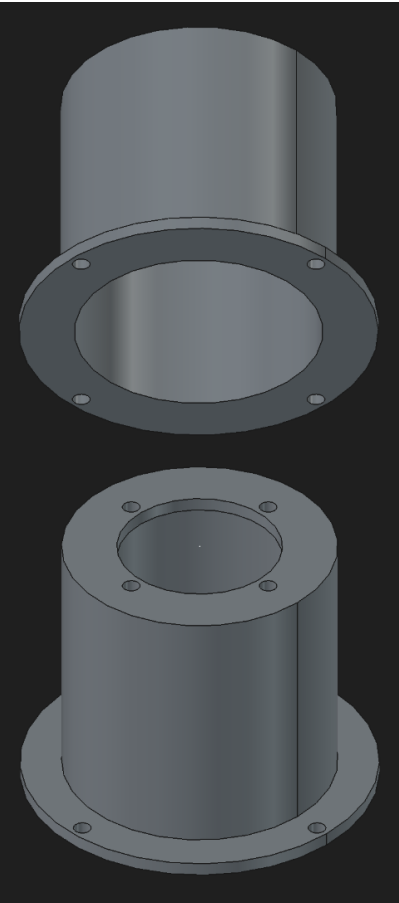
3300uF capacitor	1	
Schottky-diode IN5817	6	
Resistors: • 2x 1.0 kΩ • 1x 2.7 kΩ • 1x 1.8 kΩ	4	

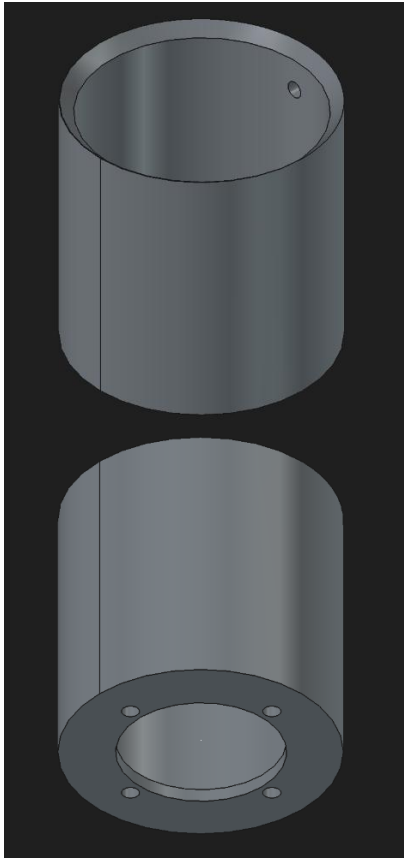
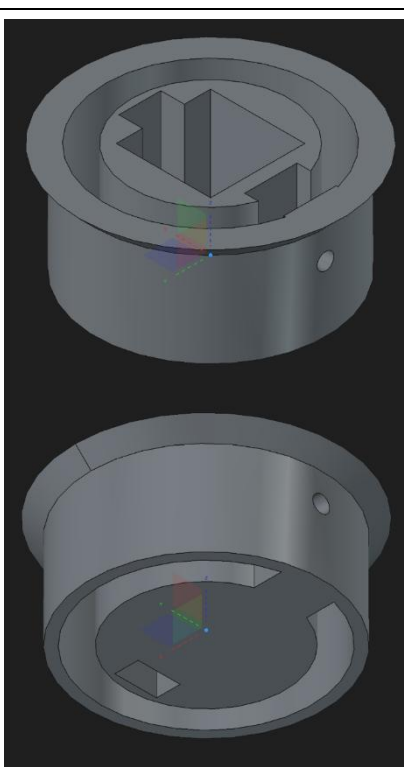
4-pole terminal block	2	
Cables	around 70	
12x8x3 Bearings	4	

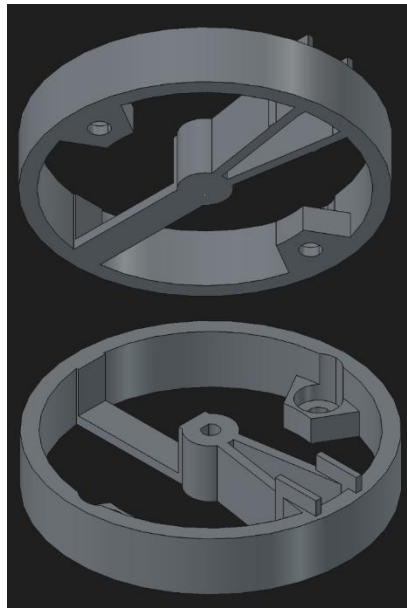
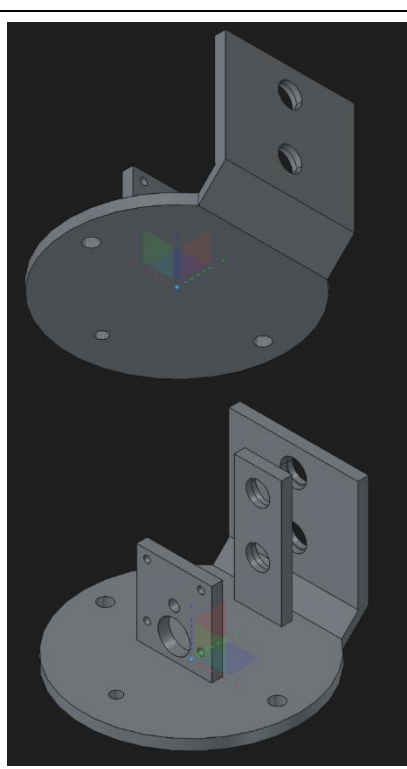
M6 Bolts	10	
M6 Nuts	10	
M3 Bolts	4	

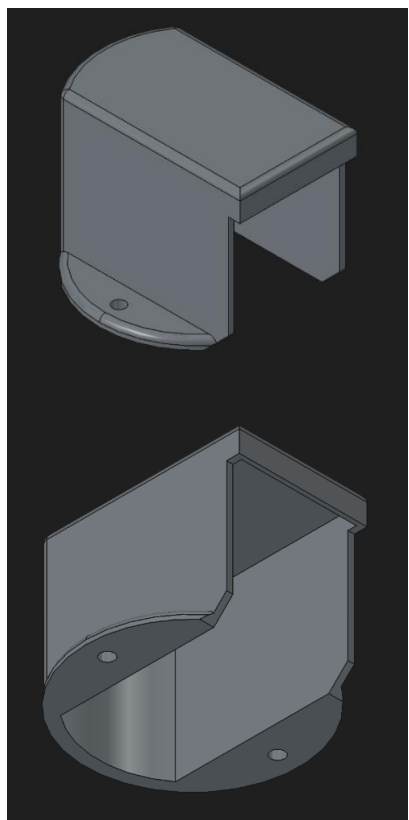
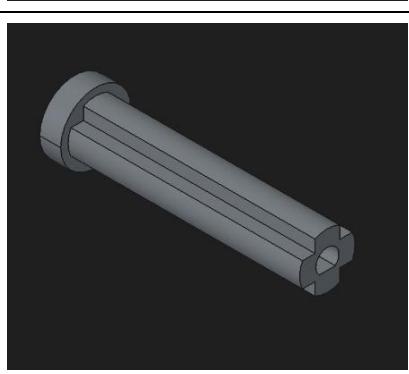
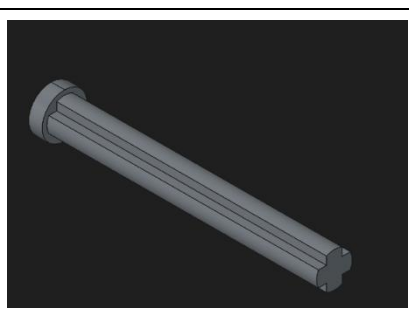
M3 Nuts	4	
USB A – microUSB cable	1	

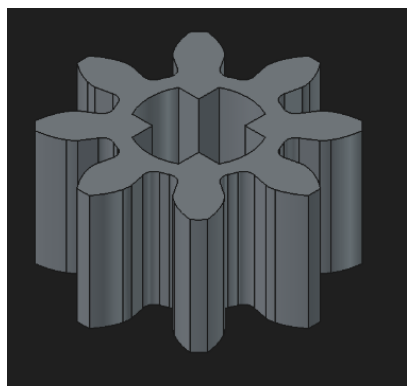
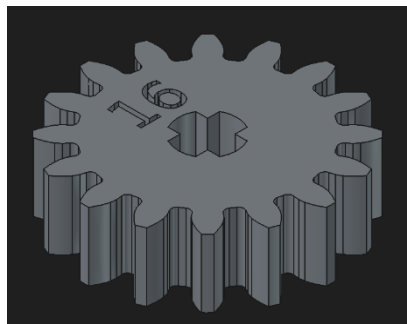
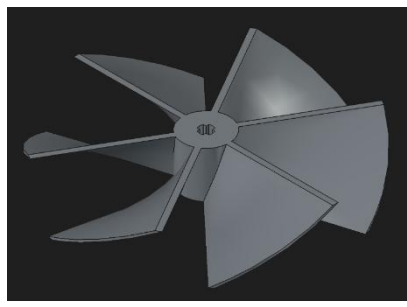
3.1.4. 3D-printed parts

Image	Description
	Wind turbine tower base

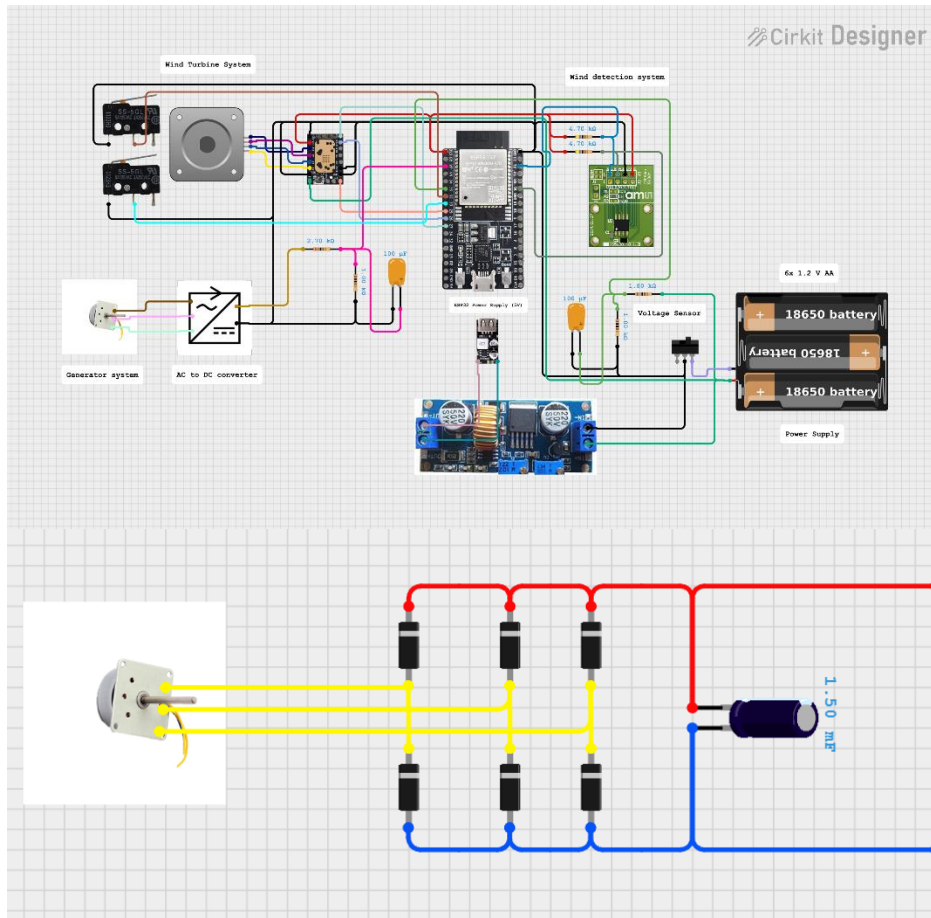
		Wind turbine tower top
		Stepper motor station

		Nacelle station
		1:2 gearbox in the nacelle

		Nacelle cover
		Gearbox bottom shaft
		Gearbox top shaft

		Gearbox bottom gear wheel
		Gearbox top gear wheel
		Rotor

3.1.5. Wind turbine and Full Wave Rectifier circuit diagram



3.2. Wind direction sensor / Wind vane

3.2.1. Description

The wind direction sensor consists of a magnetic encoder, a magnet, bearings and 3D-printed components such as the tail, shaft and housing. The magnet is attached to the wind vane's shaft, allowing the magnetic encoder to determine wind direction. The measured data is transmitted to the microcontroller via cables.

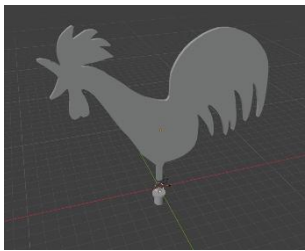
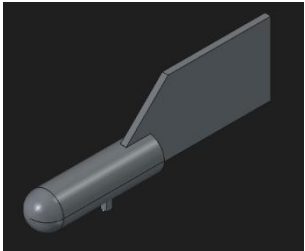
3.2.2. Challenges

Shaft

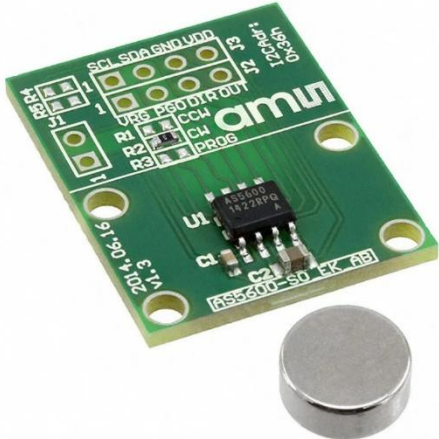
The clearance between the wind vane shaft and the bearing must be as small as possible to ensure that the magnet remains properly aligned with the magnetic encoder. We had to 3D-print the shaft multiple times due to the tolerances of the available 3D-printers.

Tail

The initial tail design was shaped like a rooster. However, this form was not suitable for accurate wind direction measurement. Eventually, the piece broke during testing. Therefore, we designed a new tail that is better suited for the wind vane.

Old tail	New tail
	

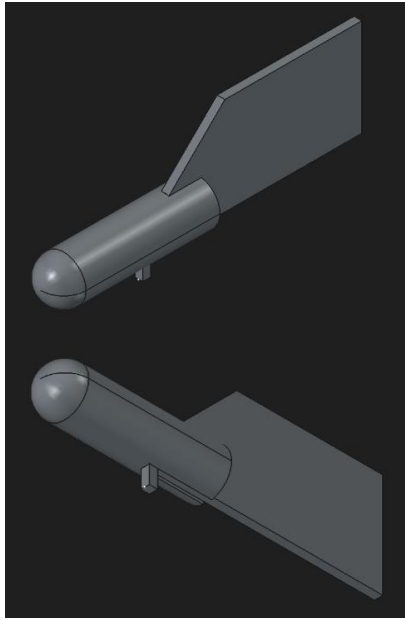
3.2.3. Components

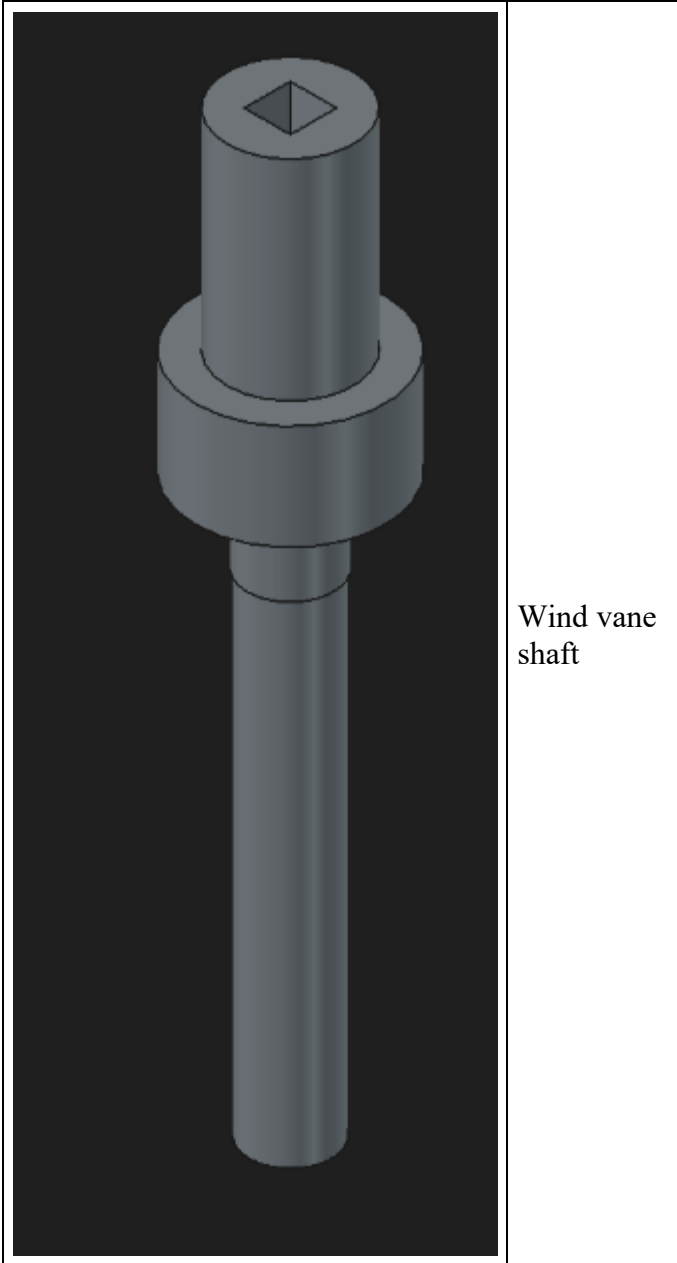
Component	Amount	Image
AS5600 magnetic encoder	1	

Resistors: 2x 4.7 kΩ	2	
608 2RS Bearings	1	
M6 Bolts	2	

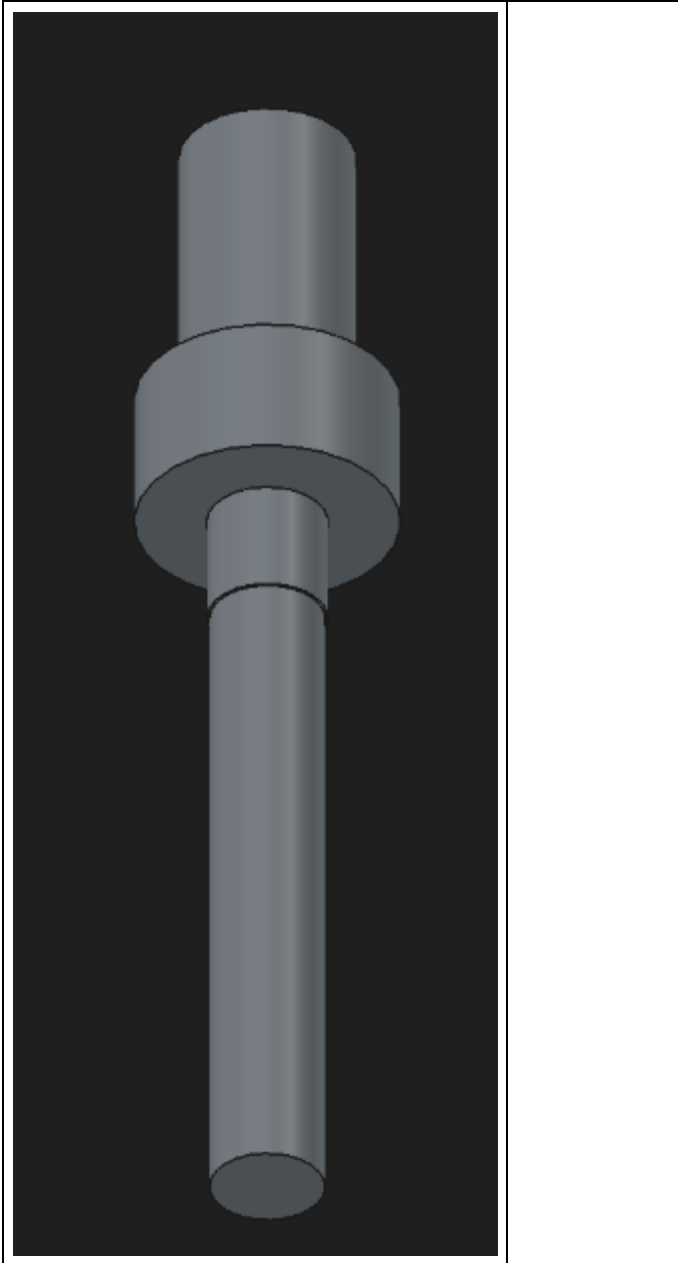
M6 Nuts	2	
---------	---	---

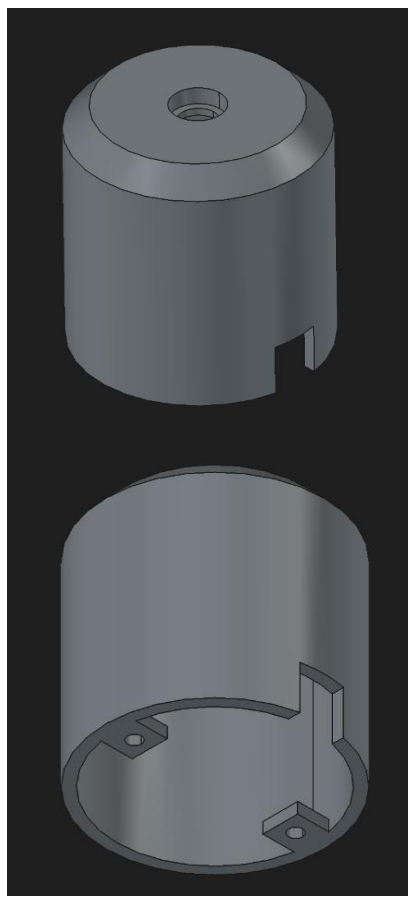
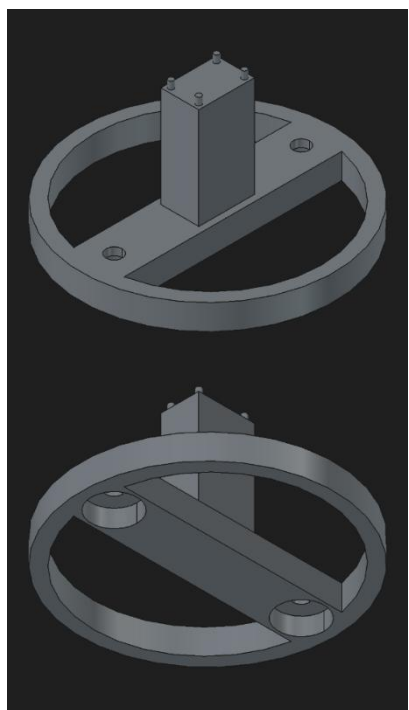
3.2.4. 3D-printed parts

Image	Description
	Wind vane tail



Wind vane
shaft



	Wind direction sensor case
	Wind direction sensor case base

3.3. Monitoring system

3.3.1. Description

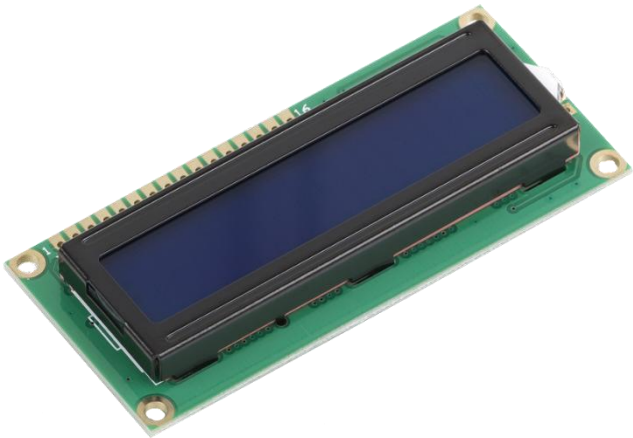
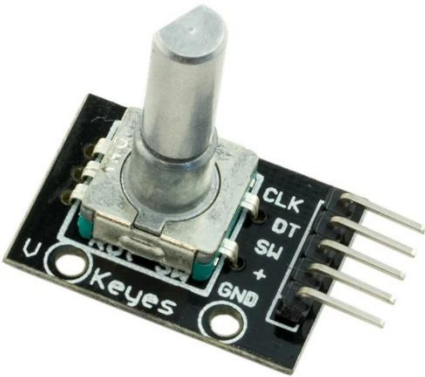
The monitoring system consists of a potentiometer, an LCD-screen, a rotary encoder, an RGB LED, an ESP32 microcontroller, and a 3D-printed case. The system fetches wind turbine sensor data from a dedicated server over the network and displays it on the screen along with additional information about the turbine. The RGB LED turns green when the wind turbine generates power, i.e., when the voltage sensor measures a value greater than 0 V. The system is powered by a power bank.

3.3.2. Challenges

The biggest challenge in the implementation of the wind turbine monitoring system was related to the software/code: fetching data from the cloud via HTTP requests noticeably reduced the performance of the graphical interface. As a solution, the ESP32's built-in FreeRTOS functionality was used to isolate the data requests from the main application logic.

3.3.3. Components

Component	Amount	Image
ESP32 XXSR69	1	

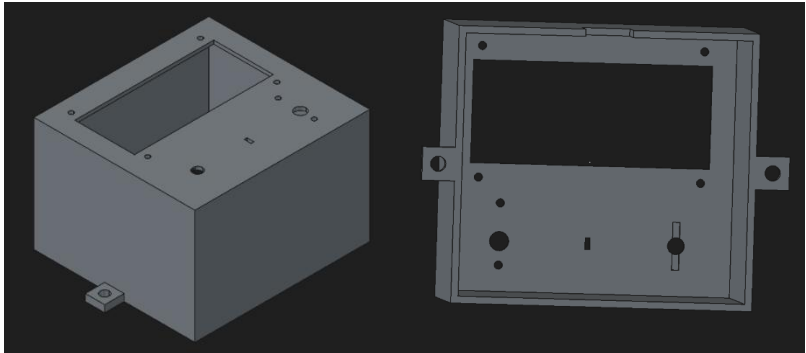
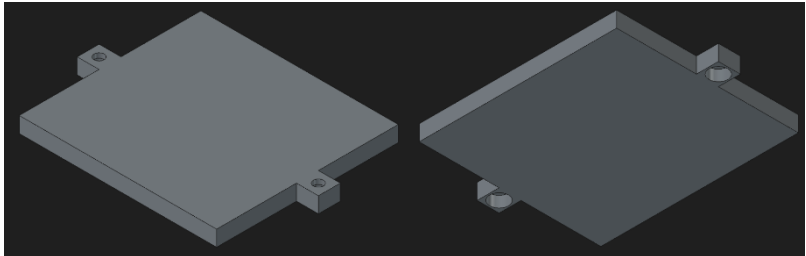
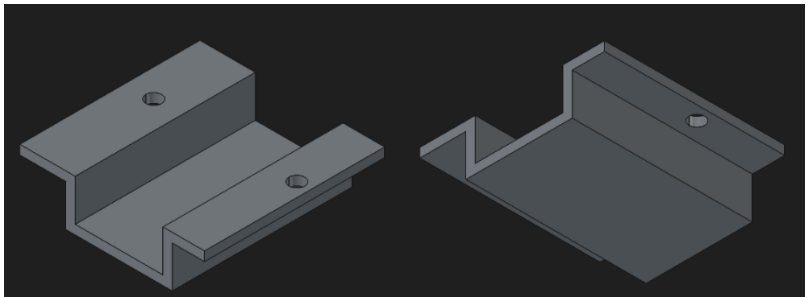
LCD screen	1	
Potentiometer	1	
Rotary encoder	1	

RGB LED	1	
Resistors: 3x 220 Ω	3	
Cables	n. 20	

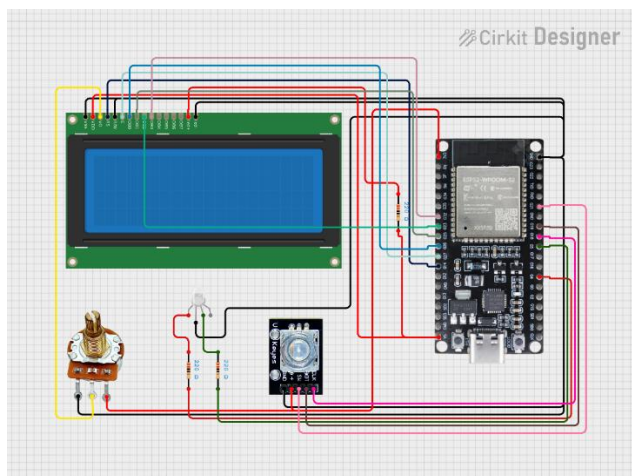
M6 Bolts	2	
M6 Nuts	2	
M3 Bolts	5	

M3 Nuts	5	
Power bank	1	
USB A - USB C cable	1	

3.3.4. 3D-printed parts

Image	Description
	Monitoring system case top
	Monitoring system case base
	Rotary encoder bracket

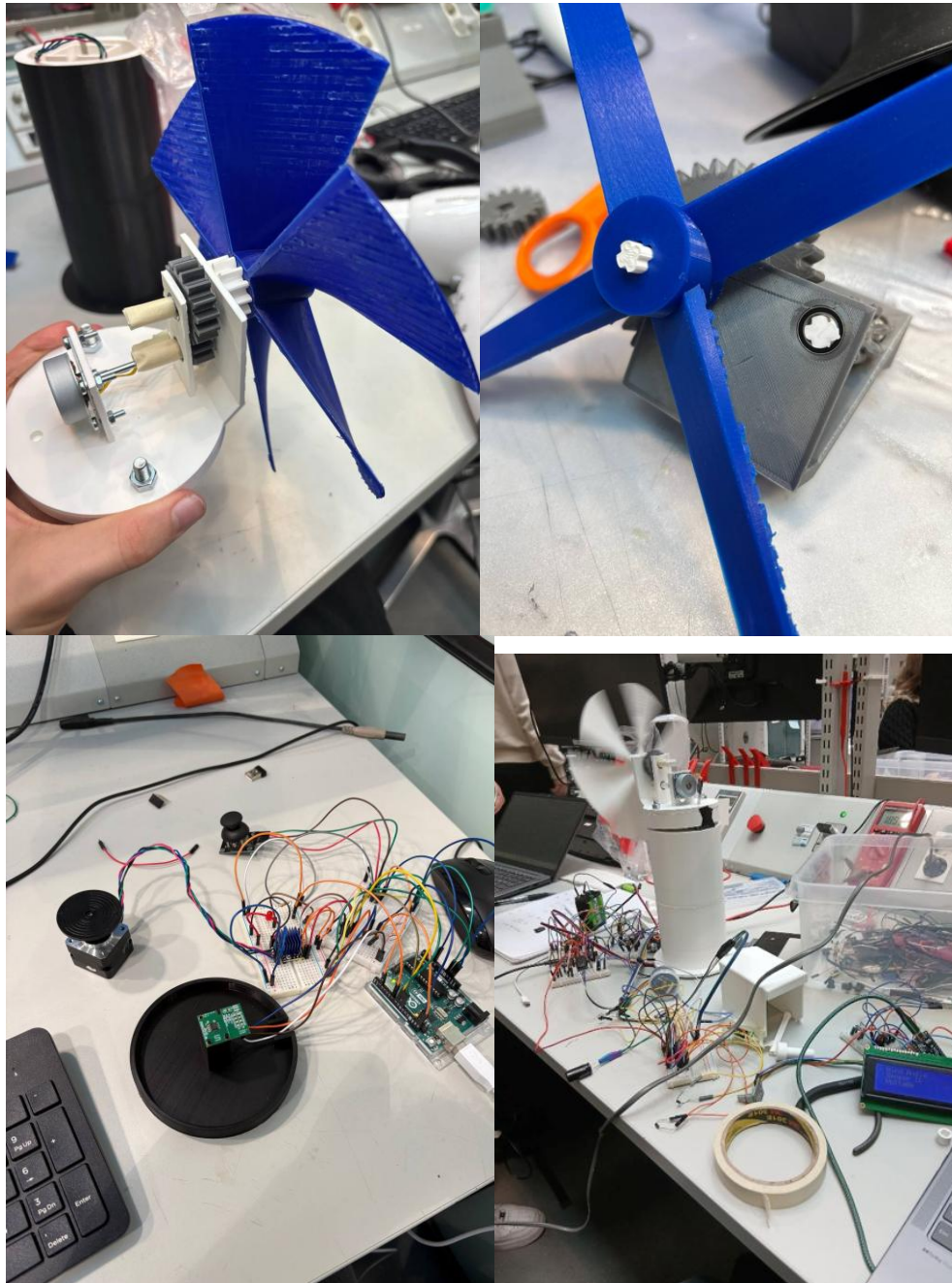
3.3.5. Circuit diagram



3.4. Wind turbine box

The electronic components of the wind turbine and wind vane are housed in a wooden box on which the turbine is mounted. All components are connected through a dedicated hole in the box.

3.5. Development pictures



4. Result

4.1. Images



4.2. Overview video

<https://youtu.be/IGIYgQ5NQMg>

5. Commented code

Wind turbine code + API

GitHub: <https://github.com/tsvetomir-zhivkov/A.L.B.E.R.T..git>

Monitoring system code

```
#include <Arduino.h>
#include <WiFi.h>
#include <HttpClient.h>
#include <LiquidCrystal.h>

LiquidCrystal lcd(14, 27, 26, 25, 33, 32); //RS=14, Enable=27, D4=26, D5=25,
D6=33, D7=32; LCD-näytön PIN-konfiguraatio

const int RED_PIN = 4;          //LED-valon PIN-konfiguraatio
const int GREEN_PIN = 5;

const int CLK_PIN = 18;         //Pulssianturin PIN-konfiguraatio: liike=18,
suunta=19, kytkin=21
const int DT_PIN = 19;
const int SW_PIN = 21;

int lastCLK = 0; //Liikemuuttujan alustus

int menuIndex = 0; // ">"-valitsimen paikkaa seuraava muuttuja
int selectedOption = 0; //Kytkinpainikkeella valittu vaihtoehto
bool inMenu = true; //Muuttuja, joka kertoo, ollaanko päävalikossa

unsigned long lastPress = 0; //Kytkinpainikkeen ajastin / Tietotyyppi valittu
millis()-funktion perusteella
const unsigned long buttonDebounce = 200; //Kahden painalluksen minimiväli
200 ms; vakauttaa kytkimen toimintaa

unsigned long lastUpdate = 0; //Näytön päivityksen ajastin
const unsigned long updateInterval = 1500; // Mittausdatan piirtotaajuus;
estää ruudun häiritsevän tiheän vilkunnan

const char* ssid = "A16"; //Wifi-komentojen edellyttämä tietotyyppi 'char*'
const char* password = "*****";

String turbineInfo[3]; //Turbiinin perustiedot
float voltage;
int windReading;
int battery;

//TÄRKEITÄ FUNKTIOITA//

void sensorTask(void *pvParameters) { //Hyödynnetään ESP32:n toista
ydintä / luodaan sensoridatan lukemista varten oma 'tehtävä'
    while(true) {
```



```

    voltage = getMeasurement(3).toFloat(); //getMeasurement tekee http-
kutsun saadakseen generaattorin jännitteen; nyt se ei sotke itse
käyttöliittymää
    windReading = getMeasurement(1).toInt();
    battery = getMeasurement(4).toInt();

    vTaskDelay(1500 / portTICK_PERIOD_MS); //Rajoittaa http-kutsujen määrää
}

}

String getTurbineData() { //Hakee turbiinin perustiedot; ajetaan pelkästään
järjestelmän käynnistyksessä

    HTTPClient http; //Alustetaan HTTP-olio, jolla on käytössään
http-yhteyden kannalta oleellisia funktioita
    http.begin("https://albert2026.azurewebsites.net/api/wind_turbine
s/4"); //Alustaa yhteyden

    int httpCode = http.GET(); //Pyytää palvelimelta dataa

    if (httpCode == HTTP_CODE_OK) {

        String payload = http.getString(); //Tallentaa palvelimelta
ladatun merkkijonon paikalliseen muuttujaan

        http.end(); //Sulkee yhteyden palvelimeen

        //DATALEIKKURI//

        String key = "\"name\":\":"; //Alustaa avaimen, jolla
leikkauskohta löydetään
        int start = payload.indexOf(key); //Alustaa muuttujan, joka
saa arvokseen leikkauskohdan aloitusindeksiin

        if (start != -1) { //indexOf-funktio palauttaa arvon -1, jos
avainta ei löydy
            start += key.length();
            int end = payload.indexOf("\"", start); //Lopettaa
näytteenoton heittomerkkiin
            if (end != -1)
                turbineInfo[0] = payload.substring(start, end);
//Lisää leikatun datan globaaliin listaan
        }

        key = "\"height\":\":"; //Turbiinin korkeus
        start = payload.indexOf(key);

        if (start != -1) {
            start += key.length();
            int end = payload.indexOf("\"", start);
            if (end != -1)
                turbineInfo[1] = payload.substring(start, end);
        }

        key = "\"rotor_diameter\":\":"; //Roottorin halkaisija
        start = payload.indexOf(key);

```

```

        if (start != -1) {
            start += key.length();
            int end = payload.indexOf("\"", start);
            if (end != -1)
                turbineInfo[2] = payload.substring(start, end);
        }
    }

    http.end();
    return ""; //Varmistaa, että virhetilanteessakin funktiolla on
palautusarvo
}

String getMeasurement(int sensorID) { //Mittau tulosten nouto palvelimelta;
jokaisella sensorilla on oma hakemisto

    HTTPClient http;
    http.begin("https://albert2026.azurewebsites.net/api/wind_turbine
s/4/sensors/" + String(sensorID) + "/logs" );
    http.setTimeout(500); //Rajoittaa sitä, kauanko järjestelmä
yrittää saada palvelimeen yhteyden

    int httpCode = http.GET();

    if (httpCode == HTTP_CODE_OK) {

        String payload = http.getString();

        http.end();

        //DATALEIKKURI//

        String measurement = "";
        String key = "\"measurement\": \""; //Ks. getTurbineData-
funktion kommentit; identtinen toteutus
        int start = payload.indexOf(key);

        if (start != -1) {
            start += key.length();
            int end = payload.indexOf("\"", start);
            if (end != -1)
                measurement = payload.substring(start, end);
        }

        return measurement;
    }

    http.end();
    return "";
}

String windDirection() { //Konvertoi tuulen suunnan asteissa ilmansuunnaksi
    if (windReading >= 338 || windReading < 23) {
        return "N";
    }
}

```

```

else if (windReading >= 23 && windReading < 68 ) {
    return "NE";
}
else if (windReading >= 68 && windReading < 113 ) {
    return "E";
}
else if (windReading >= 113 && windReading < 158 ) {
    return "SE";
}
else if (windReading >= 158 && windReading < 203 ) {
    return "S";
}
else if (windReading >= 203 && windReading < 248 ) {
    return "SW";
}
else if (windReading >= 248 && windReading < 293 ) {
    return "W";
}
else if (windReading >= 293 && windReading < 338 ) {
    return "NW";
}
return "";
}

void drawMenu() { //Valikon piirtoasusta vastaava funktio

    lcd.clear(); //Tyhjentää ruudun

    lcd.setCursor(0,0); //Asettaa piirturin 1. riville
    if (menuIndex == 0) { //Piirtää osoittimen ">" menuIndex-muuttujan
osoittamalle riville
        lcd.print("> ");
    }
    else {
        lcd.print("  ");
    }
    lcd.print("Turbine");

    lcd.setCursor(0,1); //Asettaa piirturin 2. riville
    if (menuIndex == 1) {
        lcd.print("> ");
    }
    else {
        lcd.print("  ");
    }
    lcd.print("Wind Direction");

    lcd.setCursor(0,2);
    if (menuIndex == 2) {
        lcd.print("> ");
    }
    else {
        lcd.print("  ");
    }
    lcd.print("Generator");
}

```

```

    lcd.setCursor(0,3);
    if (menuIndex == 3) {
        lcd.print("> ");
    }
    else {
        lcd.print(" ");
    }
    lcd.print("Battery");
}

void drawOption() { //Kytkinpainikkeelle valitun vaihtoehdon piirtäminen

    lcd.clear();

    if (selectedOption == 0) { //Tarkistaa selectedOption-muuttujan tilan ja
piirtää sen mukaan halutun mittausdatan
        lcd.setCursor(0,0);
        lcd.print("Turbine: " + turbineInfo[0]);
        lcd.setCursor(0,1);
        lcd.print("Height: " + turbineInfo[1] + " cm");
        lcd.setCursor(0,2);
        lcd.print("Rotor: " + turbineInfo[2] + " cm");

    }

    else if (selectedOption == 1) {

        lcd.setCursor(0,0);
        lcd.print("Wind Direction:");
        lcd.setCursor(0,1);
        lcd.print(windDirection());
    }

    else if (selectedOption == 2) {

        lcd.setCursor(0,0);
        lcd.print("Generator(V):");
        lcd.setCursor(0,1);
        lcd.print(voltage);
    }

    else if (selectedOption == 3) {

        lcd.setCursor(0,0);
        lcd.print("Battery(%):");
        lcd.setCursor(0,1);
        lcd.print(battery);
    }
}

void setup() {

    lcd.begin(20, 4);    //Alustaa LCD-näytön / 20 saraketta ja 4 riviä

    WiFi.begin(ssid, password); //Yhdistää annettuun Wifiin

```

```

    while (WiFi.status() != WL_CONNECTED) { //Odottaa, että yhteys muodostuu;
turbiinin perustietojen nouto vaatii yhteyden
        delay(500);
    }

    pinMode(RED_PIN, OUTPUT); //Määrittää punaisen ledin jännitteen
ulostuloksi
    pinMode(GREEN_PIN, OUTPUT);
    pinMode(CLK_PIN, INPUT_PULLUP); //Määrittää pulssianturin signaalin
lähteeksi ja kytkee siihen sisäisen ylösvetovastuksen
    pinMode(DT_PIN, INPUT_PULLUP);
    pinMode(SW_PIN, INPUT_PULLUP);

    Serial.begin(115200); //Alustaa kommunikaation koneen ja ESP:n välille

    lastCLK = digitalRead(CLK_PIN); //Lukee pulssianturin alkutilan

    drawMenu(); //Piirtää LCD-näytölle alkuvalikon
    getTurbineData(); //Hakee tuuliturbiinin perustiedot

    xTaskCreatePinnedToCore( /*Luo 'tehtävä'-silmukan, joka toimii
riippumattomasti muun toiminnallisuuden taustalla. Tämän käyttö oli tekoälyn
ehdotus;
                                koin sen konseptina äärimmäisen hyödylliseksi
ja painoin sen mieleeni helpottamaan vastaavien projektien toteutusta
tulevaisuudessa */
        sensorTask,          // 'tehtävä'-funktio
        "LED Controller", // Nimi
        4096,                // Muistin varaus
        NULL,                // Parametrit
        1,                   // Suoritusprioriteetti
        NULL,                // Käsittelijä
        0                     // Ydin, jolle tehtävä annetaan; ESP:illä on 2
ydintä(!)
    );
}

void loop() {

    if (voltage < 0.01) { //LED:in käyttöliittymä; jos jännite on nolla,
niin palaa punainen ledi
        digitalWrite(RED_PIN, HIGH);
        digitalWrite(GREEN_PIN, LOW);
    } else {
        digitalWrite(RED_PIN, LOW); //Nollasta poikkeavalla arvolla
vihreä
        digitalWrite(GREEN_PIN, HIGH);
    }

    int currentCLK = digitalRead(CLK_PIN); //Lukee pulssianturin nykytilan

    if (inMenu) { //Tarkistaa ollaanko alkuvalikossa

        if (currentCLK != lastCLK) { //Tarkistaa, onko pulssianturin tila
muuttunut

```

```

        if (digitalRead(DT_PIN) != currentCLK) { //Tarkistaa anturin
pyörimissuunnan
            menuIndex++; //Valikon osoitin myötäpäivään
        }
        else {
            menuIndex--; //Valikon osoitin vastapäivään
        }

        if (menuIndex > 3) menuIndex = 0; //Jos valikon osoitin pyrkii
siirtymään ruudun ulkopuolelle, osoitin palautetaan 1. riville
        if (menuIndex < 0) menuIndex = 3;

        drawMenu(); //Piirtää alkuvalikon uudella osoittimen paikalla
    }
}

lastCLK = currentCLK; //Siirtää piirtämiseen käytetyn anturin arvon
seuraavan loop()-kierroksen vertailuarvoksi

if (digitalRead(SW_PIN) == LOW) { //Lukee pulssianturin painikkeen tilan

    if (millis() - lastPress > buttonDebounce) { //Varmistaa, että
edellisestä painalluksesta on kulunut riittävästi aikaa

        lastPress = millis(); //Päivittää juuri tehdyn painalluksen
ajankohdan muistiin

        if (inMenu) {

            selectedOption = menuIndex; //Päivittää painikkeella
valituksi vaihtoehdoksi sen, jossa osoitin painallushetkellä on
            inMenu = false; //Poistutaan alkuvalikosta
            drawOption(); //Piirretään valittu mittausdata
        }
        else {

            inMenu = true; //Jos ei olla alkuvalikossa, painallus
palauttaa käyttäjän alkuvalikkoon
            drawMenu(); //Piirretään alkuvalikko
        }
    }
}

if (!inMenu) { //Ajetaan, jos LCD-ruutu esittää valitun vaihtoehdon
mittausdataa

    if (millis() - lastUpdate > updateInterval) { //Tarkistaa, onko
ruudun edellisestä päivityksestä kulunut riittävästi aikaa

        lastUpdate = millis(); //Jos päivitys tapahtuu, "nollataan"
laskuri

        drawOption(); //Piirtää mittausdatan uudelleen
    }
}
}

```

6. API links

Domain: albert2026.azurewebsites.net (shutting down 10.06.2026)

- Information on all wind turbines - domain/api/wind_turbines
- Information on one specific wind turbine - [domain/api/wind_turbines/\(tuuliturbiinin ID\)](domain/api/wind_turbines/(tuuliturbiinin ID))
- Information on all sensors of an individual wind turbine - [domain/api/wind_turbines/\(tuuliturbiinin ID\)/sensors](domain/api/wind_turbines/(tuuliturbiinin ID)/sensors)
- Information on a specific sensor of an individual wind turbine - [domain/api/wind_turbines/\(tuuliturbiinin ID\)/sensors/\(anturin ID\)](domain/api/wind_turbines/(tuuliturbiinin ID)/sensors/(anturin ID))
- Information on measurements of all sensors of an individual wind turbine - [domain/api/wind_turbines/\(tuuliturbiinin ID\)/sensors/logs/all](domain/api/wind_turbines/(tuuliturbiinin ID)/sensors/logs/all)
- Information on a measurements of a specific sensor of an individual wind turbine - [domain/api/wind_turbines/\(tuuliturbiinin ID\)/sensors/\(anturin ID\)/logs](domain/api/wind_turbines/(tuuliturbiinin ID)/sensors/(anturin ID)/logs)

7. Improvement ideas

Generator's cables: We will use slip rings instead of traditional cables and microswitches. Thus, the nacelle of the wind turbine will be able to rotate freely without the risk of twisted wires or restricted movement.

Brakes: Brakes will be used to lock the nacelle so that it does not move due external forces (e.g., high wind).

Wind vane location: We will move the wind vane to the top of the wind turbine nacelle to improve measurement accuracy.

Charging system: The wind turbine utilizes six 1.2 V rechargeable batteries to rotate the nacelle. Without a charging system, the batteries will discharge rapidly. The improved version of the turbine will charge the batteries and operate using the generated energy.

8. Sources

https://www.mouser.com/datasheet/2/588/AS5600_UG000254_2_00-1877354.pdf?srltid=AfmBOop892_kxxUgXnjFmKxcAdpxNOvunGEGbfWT3i0n2kUvdvzuPQ5A

<https://github.com/RobTillaart/AS5600>

<https://docs.arduino.cc/learn/electronics/stepper-motors/>

<https://www.kuisma.eu/elper/6puolijohdeet/9puolijohde.html>